Introduction to LLM Agents and RAG Systems

A Study Guide for the Outskill AI Accelerator Program

1. What are LLM Agents?

LLM agents are AI systems that use large language models as a reasoning engine to autonomously decide which actions to take in order to complete a given task. Unlike simple chatbots that respond to one message at a time, agents can plan, use tools, and iterate over multiple steps until they reach a final answer.

The key capability of an agent is tool use. Tools are functions the agent can call to interact with the outside world. Common tools include web search, code execution, database queries, calculators, and API calls. The agent decides which tool to call, passes the correct inputs, and interprets the result to continue its reasoning.

2. The ReAct Framework

ReAct stands for Reasoning and Acting. It is the most widely used pattern for building LLM agents. The agent follows a loop of three steps: Thought, Action, and Observation.

- Thought: The agent reasons about what to do next given the current state.
- Action: The agent calls a tool with specific inputs.
- Observation: The agent reads the tool output and decides whether to continue or answer.

This loop repeats until the agent decides it has enough information to give a final answer. In LangGraph, create_react_agent() builds this loop automatically. You only need to provide the LLM and a list of tool functions decorated with @tool.

3. What is RAG?

RAG stands for Retrieval-Augmented Generation. It is a technique that improves LLM responses by grounding them in external documents. Instead of relying on the model's training data alone, RAG retrieves the most relevant passages from a document store and includes them in the prompt.

A typical RAG pipeline has five stages:

- Load: Read documents from a source such as a PDF, web page, or database.
- Split: Break documents into smaller chunks so embeddings capture focused meaning.
- Embed: Convert each chunk into a vector using an embedding model.
- Store: Save the vectors in a vector database such as FAISS or LanceDB.
- Retrieve: At query time, embed the question and find the most similar chunks.

The retrieved chunks are inserted into the LLM prompt as context. This allows the model to answer questions about documents it has never seen during training, with much lower hallucination rates compared to open-ended generation.

Introduction to LLM Agents and RAG Systems

A Study Guide for the Outskill AI Accelerator Program

4. FAISS Vector Store

FAISS (Facebook AI Similarity Search) is an open-source library for efficient similarity search over dense vectors. It is one of the most popular choices for local RAG pipelines because it runs entirely in memory with no server required.

In LangChain, FAISS.from_documents() takes a list of document chunks and an embedding model, computes embeddings for each chunk, and stores them in a searchable index. The method db.as_retriever(search_kwargs={'k': 5}) returns a retriever that fetches the top-5 most similar chunks for any query.

The embedding model used in this course is sentence-transformers/all-MiniLM-L6-v2 from HuggingFace. It is a lightweight model that produces 384-dimensional vectors and runs on CPU without requiring a GPU.

5. Agentic RAG

Agentic RAG combines the document-grounding of RAG with the flexibility of an agent. The system first attempts to answer a question using the uploaded documents. If the retrieved context does not contain enough information, it falls back to the agent, which can use tools such as web search to find a current answer.

This two-step fallback pattern has several benefits:

- Accuracy: Document-sourced answers are grounded and verifiable.
- Flexibility: Questions outside the document corpus are still answered via tools.
- Control: You can see clearly which source produced the answer.

The fallback is triggered by instructing the LLM to respond with a sentinel string such as INSUFFICIENT_CONTEXT when the retrieved documents do not help. The application checks for this string and routes the query to the agent if it appears.

6. LangChain Expression Language (LCEL)

LCEL is the modern way to compose LangChain chains. It uses the pipe operator | to connect a prompt template, an LLM, and an output parser into a single chain that can be invoked with one call.

A simple LCEL chain: chain = prompt | llm | StrOutputParser(). To run it: result = chain.invoke({'variable': value}). For multi-step pipelines where each step produces a named output used by later steps, invoke each chain separately and pass the accumulated results as a dictionary.

LCEL replaced the older LLMChain, SimpleSequentialChain, and SequentialChain classes. The new approach is more composable, easier to debug, and supports streaming out of the box.